

铂贝学院
Bobei College

Prometheus自主无人机
Aruco码检测与追踪

Prometheus
自主无人机软件平台

主讲：戚煜华

嘉宾：金忍

铂贝学院
Bobei College

今日直播内容

- Aruco码检测与位姿估计
- 使用Prometheus控制接口进行目标跟踪
- 自主降落Gazebo仿真演示
- Prometheus二次开发指导、科研闲聊

- 授课过程中请尽量发送和课程相关弹幕，看到就会回答
- 与今日直播内容无关的弹幕尽量在最后的聊天环节提问

个人简介

铂贝学院
Bobei College

戚煜华

- **中山大学 系统科学与工程学院 博士后**

  目前有数学、计算机科学与技术、信息与通信工程三个专业。

  欢迎报考、就业！

- Prometheus开源项目创始人

主要研究方向为：无人机控制、自主避障等。

目前探索方向：协同SLAM、基于深度强化学习的路径规划



嘉宾介绍

铂贝学院
Bobei College

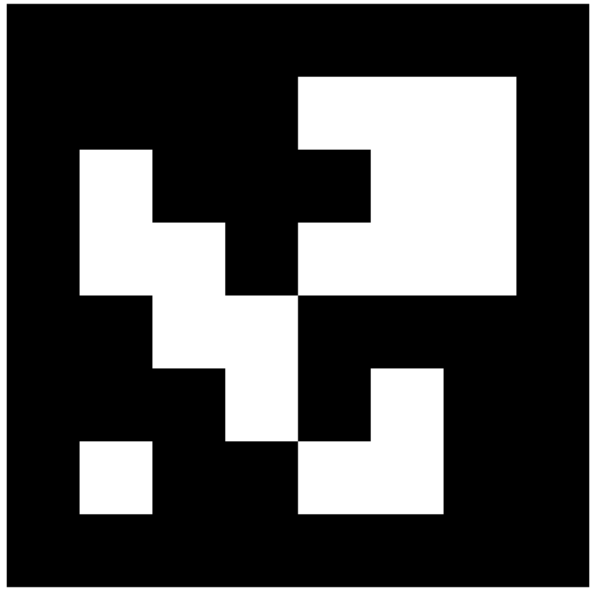
金忍，北京理工大学博士，多年从事机器视觉、无人机视觉的开发及教学工作，曾作为视觉组队长参加2017与2020年穆罕默德·本·扎耶德国际机器人挑战赛（MBZIRC），两次获得冠军。目前在Journal of Field Robotics等SCI期刊上发表论文3篇，会议论文2篇。擅长数据驱动算法编写、性能优化、实时计算等，研究兴趣主要包括深度学习，目标检测与跟踪，迁移学习，目标定位与姿态估计。



铂贝学院 Bobei College

什么是Aruco码?

- 最容易识别的目标之一



DICT_6X6_250_0

Aruco码能做什么?

- 搜索任务、自主降落等辅助标识
- 替代复杂任务中较难识别的目标（短期替代、长期替代）
- SLAM中的地标
- 反解无人机位置、实现定点
- …

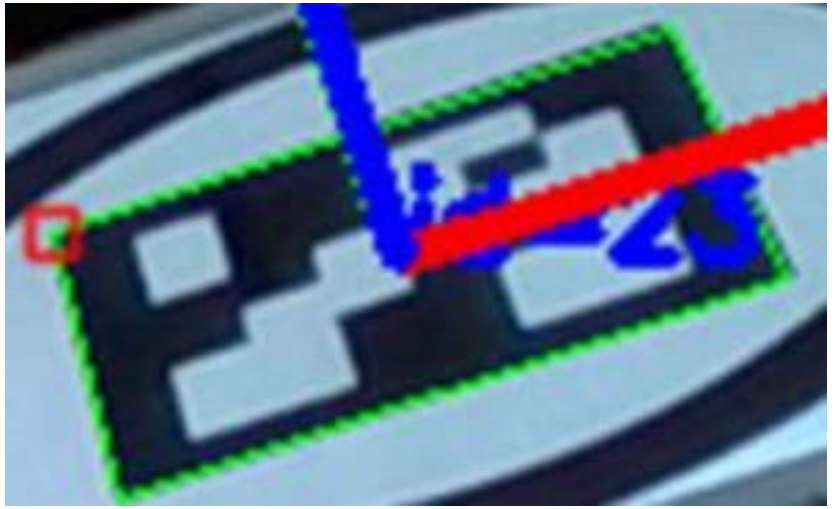
铂贝学院
Bobei College

1 候选框检测

Aruco码外部由正方形框构成，因此，首先肯定要检测出候选框，由于**透视变换**的影响，导致正方形在图像上的投影可以为**任意4边形**，唯一可知的性质就是，这个4边形一定是具有**凸性**的，代码也是利用这个性质进行筛选的。

候选4边形检测算法分为三个阶段：

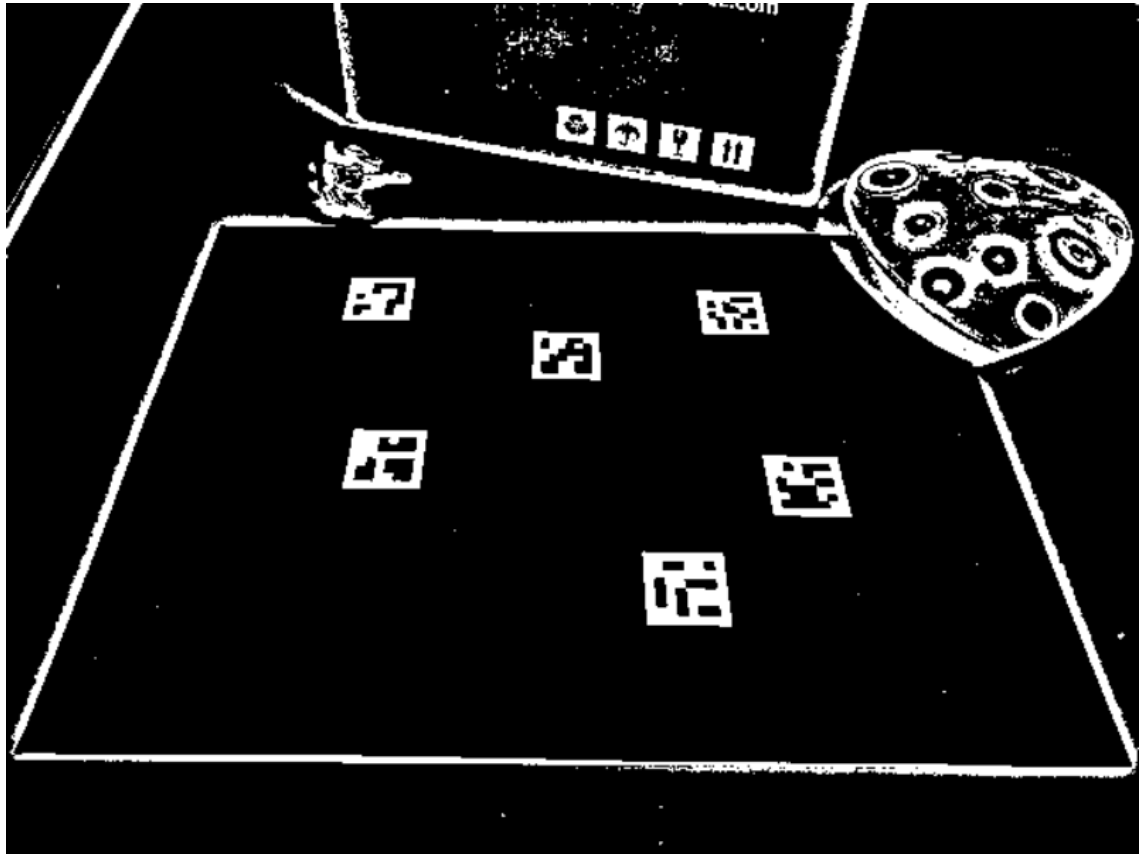
- 凸四边形检测
- 角点排序
- 去除相似框



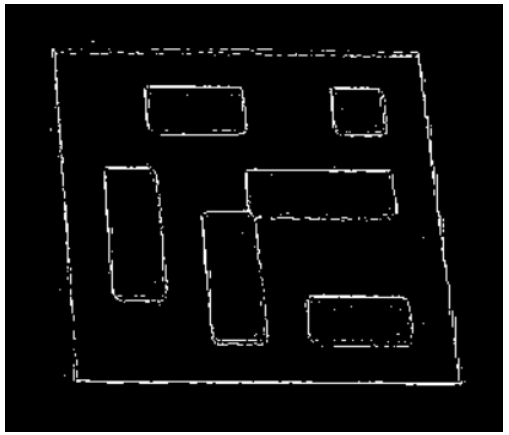
铂贝学院
Bobei College

1.1 凸四边形检测

记检测过程的第一步之一是对输入图像进行自适应二值化处理



adaptiveThreshWinSizeMin控制的是最小窗口，
adaptiveThreshWinSizeMax控制的是最大窗口，
adaptiveThreshWinSizeStep控制窗口步长，



如果二维码太大，则窗口小可能会“破坏”标记边界，导致无法检测到它。
如果二维码太小，则窗口小会产生相同的效果，并且还会降低性能。而且，该过程将趋向于全局阈值化，从而失去了自适应收益。

铂贝学院
Bobei College

1.1 凸四边形检测

基于规则的凸四边形检测

- 特别大特别小的轮廓一般都不会是目标，最小周长阈值为minPerimeterPixels = minPerimeterRate * max(rows, cols)，最大周长阈值为maxPerimeterPixels = maxPerimeterRate * max(rows, cols)
- 对这个轮廓进行多边形逼近approxPolyDP(contours[i], approxCurve, double(contours[i].size()) * polygonalApproxAccuracyRate, true);。double(contours[i].size()) * polygonalApproxAccuracyRate是对应的逼近精度，简单来说，越大的轮廓具有更大的逼近阈值。
- 轮廓逼近点必须为4个点（4边形），且这个4边形一定是凸的，否则这个轮廓就一定不是目标轮廓。
- 4边形的4个角点之间的最小距离minDistSq必须大于轮廓周长*minCornerDistanceRate，否则就不是候选。
- 判断4边形是否存在一个角点离图像的边界非常近，打个比方，角点的坐标为（1,1），与图像边界非常近，如果有那么这个就扔掉，边界距离阈值为minDistanceToBorder。

铂贝学院
Bobei College

1.2 角点排序

角点排序使用函数_reorderCandidatesCorners(candidates);，保证角点的顺序是顺时针方向，仅此而已。

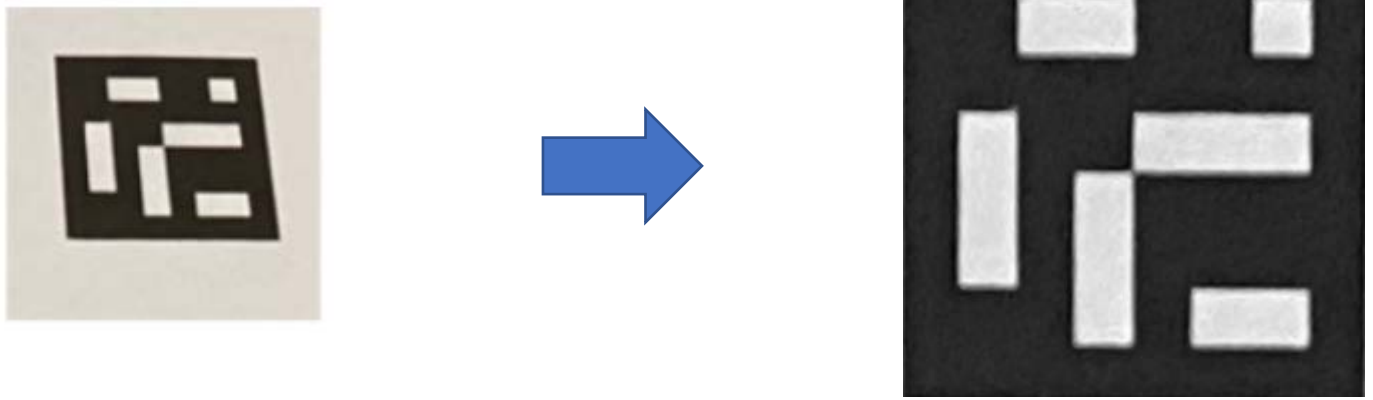
1.3 去除相似4边形

对于两个4边形，其4个角点之间的最短平均距离如果小于两个矩形的最小周长*minMarkerDistanceRate，那么就认为这两个矩形是相似的。矩形的周长就是轮廓像素个数，两个相似4边形，周长小的会被扔掉。

铂贝学院
Bobei College

2 二维码识别（比特提取）

2.1 对每个四边形利用getPerspectiveTransform和warpPerspective进行透视变换，透视变换的目标为方形，其边长为perspectiveRemovePixelPerCell*比特边长



例如，假设我们正在处理5x5位的标记和1位的边框大小。然后，每个维度的单元/位总数为5 + 2 * 1 = 7（边界必须计数两次）。单元格总数为7x7。 如果PerspectiveRemovePixelPerCell的值为10，则所获得图像的大小将为10 * 7 = 70-> 70x70像素。

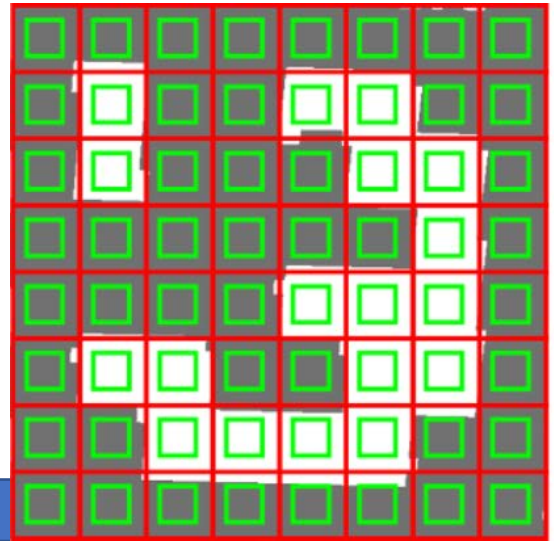
铂贝学院
Bobei College

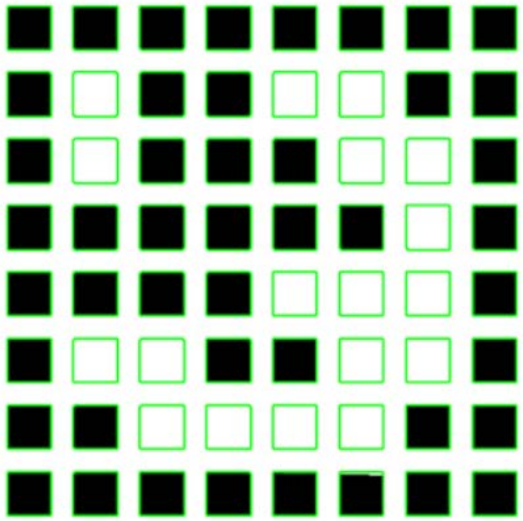
2 二维码识别（比特提取）

2.2 判断四边形内部是否为全黑或全白。先移除边框，然后利用meanStdDev计算区域的均值和方差，如果方差小于minOtsuStdDev，说明是全黑或全白，如果均值大于127，则认为是全白，否则全黑。

2.3 利用大津阈值对透视变换后的图像做个二值化threshold(resultImg, resultImg, 125, 255, THRESH_BINARY | THRESH_OTSU);

2.4 因为已经知道码标的边长的比特数，那么就直接对二值化图像提取出对应的图像块。图像块的四周靠近边界部分肯定有噪声，所以四周边界部分扔掉，那么这个边界宽度为cellMarginPixels = perspectiveRemoveIgnoredMarginPerCell * perspectiveRemovePixelPerCell





铂贝学院
Bobei College

3 对检测出的二维码候选进行过滤

检测结果中可能会出现两个码标ID一样，且其中一个码标在另一个码标里面，属于包含关系。

那么针对这个情况就要先找出具有包含关系的矩形，判断ID是否一样，一样就删掉。

4 角点修正

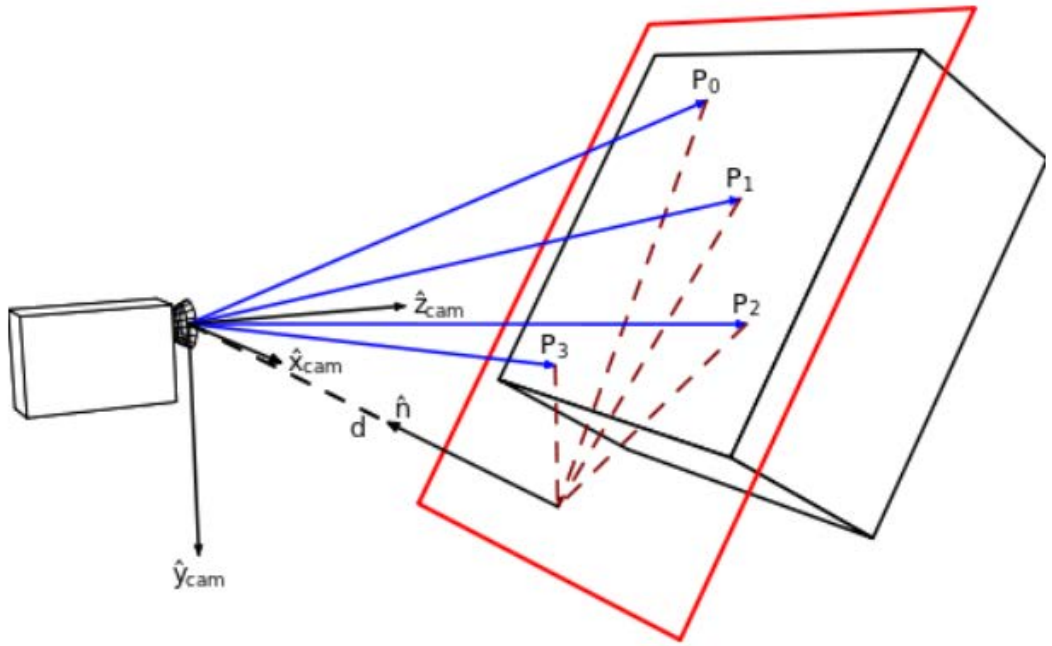
前面步骤检测出的角点都是像素级的，用于后续位姿估计时候可能会有误差，因此检测出角点之后，需要对其进行细化，得到亚像素角点，细化方法有两种，分别为：角点细化（CORNER_REFINE_SUBPIX）和拟合直线细化（CORNER_REFINE_CONTOUR）。细化方法的选择，指定参数cornerRefinementMethod即可，算法默认是不细化的。

铂贝学院
Bobei College

5 姿态估计

PnP（Perspective-n-Point）是求解3D到2D点对运动的方法。它描述了已知n个3D空间点及其图像上的投影位置时，如何估计相机的位姿。

PnP问题有很多种求解方法，例如，用3对点估计位姿的P3P、直接线性变换（DLT）、EPnP、UPnP，等等。此外，还能用非线性优化的方法，构建最小二乘问题并迭代求解，也就是Bundle Adjustment。



铂贝学院 Bobei College

Prometheus中的二维码识别与封装

Aruco检测代码位置：Modules\object_detection\aruco_det.cpp

```
<!-- 启动二维码检测算法 -->
<node pkg="prometheus_detection" type="aruco_det" name="aruco_det" output="screen">
    <!-- 相机话题 -->
    <param name="camera_topic" type="string" value="/prometheus/sensor/monocular_down/image_raw" />
    <!-- 相机参数 -->
    <param name="camera_parameters" type="string" value="$(find prometheus_gazebo)/config/camera_config/gimbal_camera.yaml" />
    <!-- 输出话题1 -->
    <param name="output_pose_topic" type="string" value="/prometheus/object_detection/aruco_det" />
    <!-- 输出话题2 -->
    <param name="output_multi_pose_topic" type="string" value="/prometheus/object_detection/multi_aruco_det" />
    <!-- 输出图像话题 -->
    <param name="output_image_topic" type="string" value="/prometheus/camera/rgb/image_aruco_det" />
    <!-- 10号字典 -->
    <param name="dictionary_type" type="int" value="10" />
    <!-- 指定识别1号marker -->
    <param name="binding_id" type="int" value="1" />
    <!-- marker边长 10:8 -->
    <param name="target_marker_length" type="double" value="0.4" />
</node>
```

Launch中调用示例

铂贝学院 Bobei College

识别模块的输出

```
std_msgs/Header header

## aruco编码
int32 aruco_num

## 是否检测到目标
bool detected

## 目标位置-相机坐标系-(x,y,z)
## 从相机往前看，物体在相机右方x为正，下方y为正，前方z为正
float32[3] position

## 目标姿态-四元数-(qx,qy,qz,qw)
float32[4] orientation

## 视线角度-相机系下-(右方x角度为正,下方y角度为正)
float32[2] sight_angle
```

ArucoInfo.msg

```
std_msgs/Header header

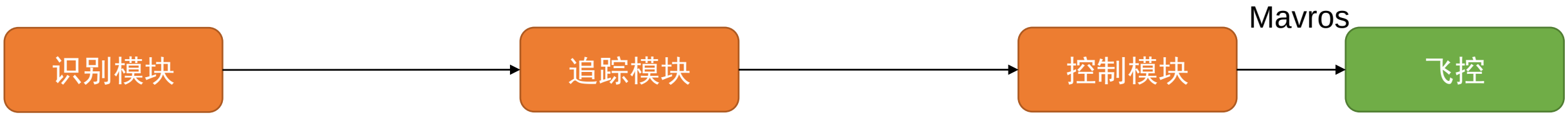
## 检测到的aruco码数量
int32 num_arucos

## 每个aruco码的检测结果
ArucoInfo[] aruco_infos
```

MultiArucoInfo.msg

铂贝学院 Bobei College

如何使用Prometheus控制接口进行目标跟踪?



/prometheus/object_detection/aruco_det

```
std_msgs/Header header

## aruco编码
int32 aruco_num

## 是否检测到目标
bool detected

## 目标位置-相机坐标系-(x,y,z)
## 从相机往前看，物体在相机右方x为正，下方y为正，前方z为正
float32[3] position

## 目标姿态-四元数-(qx,qy,qz,qw)
float32[4] orientation

## 视线角度-相机系下-(右方x角度为正,下方y角度为正)
float32[2] sight_angle
```

/prometheus/control_command

```
std_msgs/Header header

## 控制命令的编号 防止接收到错误命令， 编号应该逐次递加
uint32 Command_ID

## 消息来源
string source
## 控制命令的模式
uint8 Mode
# enum Mode 控制模式枚举
uint8 Idle=0
uint8 Takeoff=1
uint8 Hold=2
uint8 Land=3
uint8 Move=4
uint8 Disarm=5
uint8 User_Mode1=6
uint8 User_Mode2=7

## 控制参考量
## 位置参考量：位置、速度、加速度、加加速度、加加加速度
## 角度参考量：偏航角、偏航角速度、偏航角加速度
PositionReference Reference_State
```

铂贝学院 Bobei College

如何使用Prometheus控制接口进行目标跟踪?

代码位置：Modules\mission\autonomous_landing\autonomous_landing_aruco.cpp

- **订阅识别模块的输出（大部分都需要坐标转换，并进行数据处理）**

  `"/prometheus/object_detection/aruco_det"`

- **订阅无人机的状态（除了xyz信息之外，判断飞机当前模式、解锁等）**

  `"/prometheus/drone_state"`

- **发布控制指令**

  `"/prometheus/control_command"`

- **简易状态机的实现（处理好初始化、起飞、丢失目标、检测到目标、任务失败等逻辑处理），这些不同的状态会对应不同的控制策略，即不同控制指令**

- **核心状态：目标追踪**

铂贝学院
Bobei College

目标追踪

- **今天不讨论vision-based追踪，只说position-based追踪**
- **已知目标位置（一般来说只能追静止目标）**
  - 直接飞到那个位置，发送 期望目标点 至控制模块
  - 将目标位置和当前无人机位置做差，乘以系数得到 期望速度 发至控制模块（推荐）
- **已知目标位置和速度（可追动态目标）**
  - 此处不展开

铂贝学院
Bobei College

代码讲解与演示环节

- **启动脚本一: 用于检测识别算法的输出是否准确**

  Simulator\gazebo_simulator\launch_detection\sitl_aruco_detection.launch

  ```
  roslaunch Prometheus_gazebo sitl_aruco_detection.launch
  ```

- **启动脚本二: 基于Aruco码的自主降落**

  Simulator\gazebo_simulator\launch_detection\sitl_landing_on_aruco_marker.launch

  ```
  roslaunch Prometheus_gazebo sitl_landing_on_aruco_marker.launch
  ```

Prometheus控制模块

铂贝学院
Bobei College

小结

- **Aruco识别**：Modules\object_detection\aruco_det.cpp
- **追踪控制**：Modules\mission\autonomous_landing\autonomous_landing_aruco.cpp
- **启动脚本:**

  Simulator\gazebo_simulator\launch_detection\sitl_aruco_detection.launch (识别测试)

  Simulator\gazebo_simulator\launch_detection\sitl_landing_on_aruco_marker.launch （基于Aruco码的自主降落)

结束语

铂贝学院
Bobei College

- **几点建议：**
  - 任何课程、资料都只能辅助理解，能否会用，还是要看自己付出的**努力**
  - 如果希望理论有突破，那就**勤读论文**
  - 如果希望进行二次开发，那就**勤读代码**
  - 如果希望做出漂亮的实验，那就**勤动手**
- **Prometheus校园赞助计划**
  - 奖励利用Prometheus进行仿真实验并发表论文的学生
  - 奖励为Prometheus提供新功能新模块的学生开发者
- **后续直播预告**
  - 基于PX4云台的目标检测与追踪
  - Prometheus中的集群控制框架
  - ...

铂贝学院
Bobei College



Thanks for listening!